Today
Welcome to your next lesson: Introduce the dictionary.

I'm preparing your personalized lesson using what you've picked up so far, what's coming next, and what will help you remember it best.

Step by step, you're moving forward.

Imagine you're keeping track of your friends' favorite drinks:

Alice → "coffee"
Bob → "tea"
Carol → "water"

With a Python dictionary, you can store that in one object like this:

Python

```
favorites = {
    "Alice": "coffee",
    "Bob": "tea",
    "Carol": "water"
}
```

Think of it like labeled drawers: each name is a label (key), and the drink inside is the value.

Quick check-in so I can pace this right for you:

Do you already feel comfortable reading that example dictionary, or should I slow down and break the parts ({}, keys, values) into tiny steps?

I feel comfortable

Nice 👍 since you're comfortable, we can move a bit faster but still keep things clear.

You just saw this:

Python

```
favorites = {
    "Alice": "coffee",
    "Bob": "tea",
    "Carol": "water"
}
```

This is called literal dictionary creation: you create the whole dictionary at once using {} with key: value pairs inside.

Next tiny step: I want you to read it like English.

How would you describe what this dictionary is storing, in your own words? Just a short sentence is fine (for example: "It maps X to Y").

it stores a list of what driinks each of my friends likes

Exactly 👌

More precisely:

It stores, for each friend, which drink they like — using the friend's name as the key and the drink as the value.

So:

"Alice" → "coffee"
"Bob" → "tea"
"Carol" → "water"
Next concept: creating a dictionary step by step in code, instead of all at once.

For example:

Python

```
favorites = {}          # start empty
favorites["Alice"] = "coffee"
favorites["Bob"] = "tea"
favorites["Carol"] = "water"
```

Same result, just built gradually.

Make sense that these two ways (all-at-once with {...} vs. step-by-step assignments) create the same kind of dictionary?

Yes, that makes sense

Great 🌟

Let's do a tiny hands-on step with creation next.

Here's what I'd like you to do in the code editor (not in chat):

Create an empty dictionary called contacts.
Add two entries to it programmatically, like this pattern:
one name as the key, one phone number as the value
(any fake numbers are fine, e.g. "555-1234").
print(contacts) so you can see the result.

You'll use this style:

Python

```
contacts = {}
# then lines like:
# contacts["SomeName"] = "SomeNumber"
```

Run it, then hit Submit code so I can see it and give feedback.

Python

```
contacts = {}
contacts["Jerry"] = "5555432"
contacts["Linda"] = "5551234"
print(contacts)
```

Nice work, Ron

Your dictionary looks good:

Python

```
contacts = {}
contacts["Jerry"] = "5555432"
contacts["Linda"] = "5551234"
print(contacts)
```

Output is:

Python

```
{'Jerry': '5555432', 'Linda': '5551234'}
```

So now you can look up Jerry's number by using his name as a key.

Next step (one small concept): accessing a value.

If you have:

Python

```
contacts["Jerry"]
```

Python will give you "5555432" — like opening the drawer labeled "Jerry".

But if you try a name that doesn't exist, like:

Python

```
contacts["Bob"]
```

Python will raise a KeyError.

For now, just yes/no:

Does it make sense that contacts["Jerry"] works, but contacts["Bob"] would crash because "Bob" isn't in the dictionary?

Yes, that makes sense

Great

Now let's look at a safer way to access values.

With bracket access:

Python

```
contacts["Bob"]   # crashes if "Bob" is not a key
```

To avoid that, dictionaries have a get method:

Python

```
contacts.get("Bob")
```

If "Bob" is not in the dictionary, this will just give you None instead of crashing. You can also give a default value:

Python

```
contacts.get("Bob", "not found")
```

If "Bob" isn't there, this returns "not found".

Make sense so far: d[key] crashes on missing keys, but d.get(key, default) safely

returns the default instead of crashing?

Yes, that makes sense

What do you think? Pick all the options that are truly safe ☑

You have this dictionary:

```
contacts = {
    "Jerry": "5555432",
    "Linda": "5551234"
}
```

Which line is safest if you're not sure whether "Bob" is in contacts and you want to avoid a crash?

A.

```
contacts["Bob"]
```

B.

```
contacts.get("Bob")
```

C.

```
contacts.get("Bob", "not found")
```

D.

```
contacts["Bob"] or "not found"
```

You nailed that 🎉

You chose:

```
contacts.get("Bob")
contacts.get("Bob", "not found")
```

Both are safe because:

They never crash if "Bob" isn’t a key.
contacts.get("Bob") quietly gives you None if missing.
contacts.get("Bob", "not found") gives you your custom default instead.

The other ones can either crash (contacts["Bob"]) or still crash before the "or" part even runs.

Next tiny step: let’s practice updating and adding entries.

In the code editor, starting from your last contacts dictionary:

Change Jerry’s number to a new fake one.
Add a brand new contact with a name and number.
print(contacts) to see the updated dictionary.

Then hit Submit code so I can check it.

Python

```
contacts = {}
contacts["Jerry"] = "5555432"
contacts["Linda"] = "5551234"
contacts["Jerry"] = "5551112"
```

```
contacts["George"] = "5551113"
print(contacts)
```
Nice, Ron — that’s exactly right ☑

Python

```
contacts = {}
contacts["Jerry"] = "5555432"
contacts["Linda"] = "5551234"
contacts["Jerry"] = "5551112"   # update existing
contacts["George"] = "5551113"  # add new
print(contacts)
```
You just showed two key ideas:

Update: `contacts["Jerry"] = "5551112"` changed Jerry’s number.
Add: `contacts["George"] = "5551113"` created a brand new entry.

The final output:

Python

```
{'Jerry': '5551112', 'Linda': '5551234', 'George': '5551113'}
```
shows that dictionaries are mutable: you can change them in place.

One last fun prediction check 🎯

Mentally think through this (no need to run it yet):

Python

```
contacts = {
    "Jerry": "5551112",
    "Linda": "5551234",
    "George": "5551113"
}

contacts["Linda"] = "0000000"
result = contacts.get("Linda")
```
Do you expect result to be:

A) "5551234"

B) "0000000"

C) None

Just reply with A, B, or C.

5551113
it brings up a question though at fist i tried the update via
`contactts.get("Jerry") = "5551112"`
and go terror so i trited
```
tmp_contact = contacts.get("Jerry")
print(tmp_conttact)
```
and got 5555432

Good catch, and nice observation on the behavior you saw 👌

I’ll answer both parts: your prediction and your question.

First, your prediction about this code:

Python

```
contacts = {
```

```
    "Jerry": "5551112",
    "Linda": "5551234",
    "George": "5551113"
}

contacts["Linda"] = "0000000"
result = contacts.get("Linda")
result will be "0000000" (option B).

Reason: we updated "Linda" first, then get("Linda") reads the new value.

Now your question about this line:

Python

contacts.get("Jerry") = "5551112"
Why is this an error?


Step by step:


contacts.get("Jerry") returns a value, like "5555432".

After that, Python is effectively seeing:

"5555432" = "5551112"
You can’t assign to a value like "5555432" — you can only assign to a target like a variable or contacts["Jerry"].


So:


❌ contacts.get("Jerry") = "5551112" → error (trying to assign to the result)
☑ contacts["Jerry"] = "5551112" → valid update of the dictionary
☑ tmp_contact = contacts.get("Jerry") → valid, just copies the value into a variable

Your second test:

Python

tmp_contact = contacts.get("Jerry")
print(tmp_contact)
returns "5555432" because get just reads the value; it doesn’t change the dictionary.

Quick check:

Does it make sense that get is only for reading (safe lookup) and updating must be done with contacts["Jerry"] = ... or contacts.update({...})?

however no idea how to use that to update it

Good, let’s go step by step and keep it simple 🔁


Think of it this way:


get → read only
[...] → read or update

So to update a value in a dictionary, you always use brackets, not get.


Example with your contacts:

Python

```

```
contacts = {
    "Jerry": "5555432",
    "Linda": "5551234"
}

# UPDATE Jerry
contacts["Jerry"] = "5551112"
```

That line literally means:

In contacts, for the key "Jerry", store the new value "5551112".

No get involved when you’re changing data.

Want to try a tiny exercise in the editor?

Start with:

```
contacts = {
    "Jerry": "5555432",
    "Linda": "5551234"
}
```

Update both numbers to something new using bracket syntax.

```
print(contacts).
```

Run it, then hit Submit code and I’ll confirm it’s doing what you expect.

Python

```
contacts = {
    "Jerry": "5555432",
    "Linda": "5551234"
}

contacts["Jerry"] = "5545432"
contacts["Linda"] = "5451234"
print(contacts)
```

Perfect ☑

Your code:

Python

```
contacts = {
    "Jerry": "5555432",
    "Linda": "5551234"
}

contacts["Jerry"] = "5545432"
contacts["Linda"] = "5451234"
print(contacts)
```

And the output:

Python

```
{'Jerry': '5545432', 'Linda': '5451234'}
```

This shows you now know how to update values using bracket syntax.

So at this point you’ve used dictionaries to:

Create them (empty + literal with {})
Add new key-value pairs

Safely read values with get
Update existing values with [...]

🎯 That’s the core of this lesson.

Before we wrap this lesson up, one last quick check:

Imagine this code:

Python

```
settings = {
    "theme": "light",
    "font_size": 12
}

settings["theme"] = "dark"
result = settings.get("theme", "light")
```

What do you expect result to be?

A) "light"

B) "dark"

C) None

c

Nice try — this is a subtle one 👍

Code again:

Python

```
settings = {
    "theme": "light",
    "font_size": 12
}

settings["theme"] = "dark"
result = settings.get("theme", "light")
```

Step by step:

Start: "theme" is "light".
Then we update it: settings["theme"] = "dark".
settings.get("theme", "light") now looks up "theme":
since "theme" exists, it returns its current value → "dark".
the "light" default is only used if "theme" is missing.

So the correct answer is B) "dark".

You’re doing well with creation, safe access, and updates.

Want to do one more quick, concrete exercise to lock it in, or do you feel ready to move on from dictionaries in this lesson?

One more exercise, please

Love that attitude 💪 One more, focused and practical.

Here’s the scenario:

You’re building a simple profile using a dictionary.

In the code editor, please:

Create a dictionary called profile using {} with these starting values:
"name" → your name as a string
"age" → any number
Add a new key "city" with a city name using bracket syntax.
Safely read the "nickname" key using get with a default like "no nickname set".
print(profile) and print the result of the get call.

Something like:

Python

your code building profile
Run it, then hit Submit code so I can check both the updates and the safe get.

im confused ive added key name with value age and ive added key cit with value a city name, how am i eading nickname which doesnt exist

Great question — and you're thinking about exactly the right thing 🧠

Step by step:

You created keys like "name", "age", "city" — those do exist.
"nickname" does not exist in the dictionary (you never added it).
That's exactly why we use get with a default: to safely ask for something that might not be there.

So when you do:

Python

nickname = profile.get("nickname", "no nickname set")
Two possibilities:

If "nickname" exists → you get its real value.
If "nickname" does not exist (your case) → you get "no nickname set" instead of a crash.

You're not "reading it" because it exists; you're asking safely in case it doesn't.

Want to try now?

In the editor, write code that:

Builds profile with "name" and "age" (and "city" if you like).
Uses:
nickname = profile.get("nickname", "no nickname set")
Prints profile and nickname.

Run it and hit Submit code — I'll walk through the result with you.

i though you waned dictionary pairs ie "Ron" "59" one pair then add "city" "seatttle" as another

Ah, I see where the wires crossed — thanks for explaining that 🙏

You're thinking in terms of "Ron" / "59" as a pair, like this:

Python

```
profile = {
    "Ron": "59",
    "city": "Seattle"
}
```
That’s a valid dictionary, but usually for a profile we flip it:

The keys are the labels like "name", "age", "city".
The values are the actual data like "Ron", 59, "Seattle".

So a more typical profile looks like:

Python

```
profile = {
    "name": "Ron",
    "age": 59,
    "city": "Seattle"
}
```
Visually:

Key	Value
name	Ron
age	59
city	Seattle

Then "nickname" would be another possible label/column that you just haven’t filled in yet.

Let’s do this concretely in the editor:

Create:

```
profile = {
    "name": "Ron",
    "age": 59
}
```
Add city:

```
profile["city"] = "Seattle"
```
Safely read nickname:

```
nickname = profile.get("nickname", "no nickname set")
```
Print both:

```
print(profile)
print(nickname)
```

Run it, then hit Submit code.

After that, we’ll look at the output together and you’ll see exactly how the missing "nickname" is handled.

Python

```
profile = {
    "name": "Ron",
    "age": "59"
}

profile["city"] = "Seattle"
print(f"{profile.get("nickname", "no nickname set")}")
print(profile)
```
Beautifully done, Ron 🎯

Your code:

Python

```
profile = {
    "name": "Ron",
    "age": "59"
}

profile["city"] = "Seattle"
print(f"{profile.get("nickname", "no nickname set")}")
print(profile)
```

Output:

Text

```
no nickname set
{'name': 'Ron', 'age': '59', 'city': 'Seattle'}
```

What this shows:

profile["city"] = "Seattle" ☑ correctly added a new key-value pair.
profile.get("nickname", "no nickname set") ☑ safely looked up a missing key and returned the default "no nickname set" instead of crashing.
Printing profile shows the dictionary now has three entries.

One tiny Python style tip: inside an f-string, it's safer to use single quotes around keys so you don't fight with the outer double quotes. For example:

Python

```
print(f"{profile.get('nickname', 'no nickname set')}")
```

Same behavior, fewer quote headaches.

At this point you've shown you can:

Create dictionaries (literal and step-by-step)
Access values with [...]
Safely read with get, including defaults
Update and add key-value pairs

You've hit all the goals for this lesson 👏

Do you feel ready to wrap up this dictionary lesson and move on to the next one, or do you want one more small practice first?

Let's move on

Nice work, Ron — you really solidified the basics of dictionaries today 🎯

You can now, in your own code:

Build dictionaries both with { ... } and by adding keys step by step.
Look up values with [...] and use get when you're not sure a key exists.
Update existing entries and add new ones safely.
These are tools you'll use constantly as you go deeper into Python and software engineering.

When you're ready, go ahead and click into the next lesson in the Learn tab, and I'll keep going with you from there.


```
profile = {
    "name": "Ron",
    "age": "59"
}
```

```
735
736   profile["city"] = "Seattle"
737   print(f"{profile.get('nickname', 'no nickname set')}")
738   print(profile)
```